PROJE 11 / 20 · ÇALIŞAN TEMEL SÜRÜM

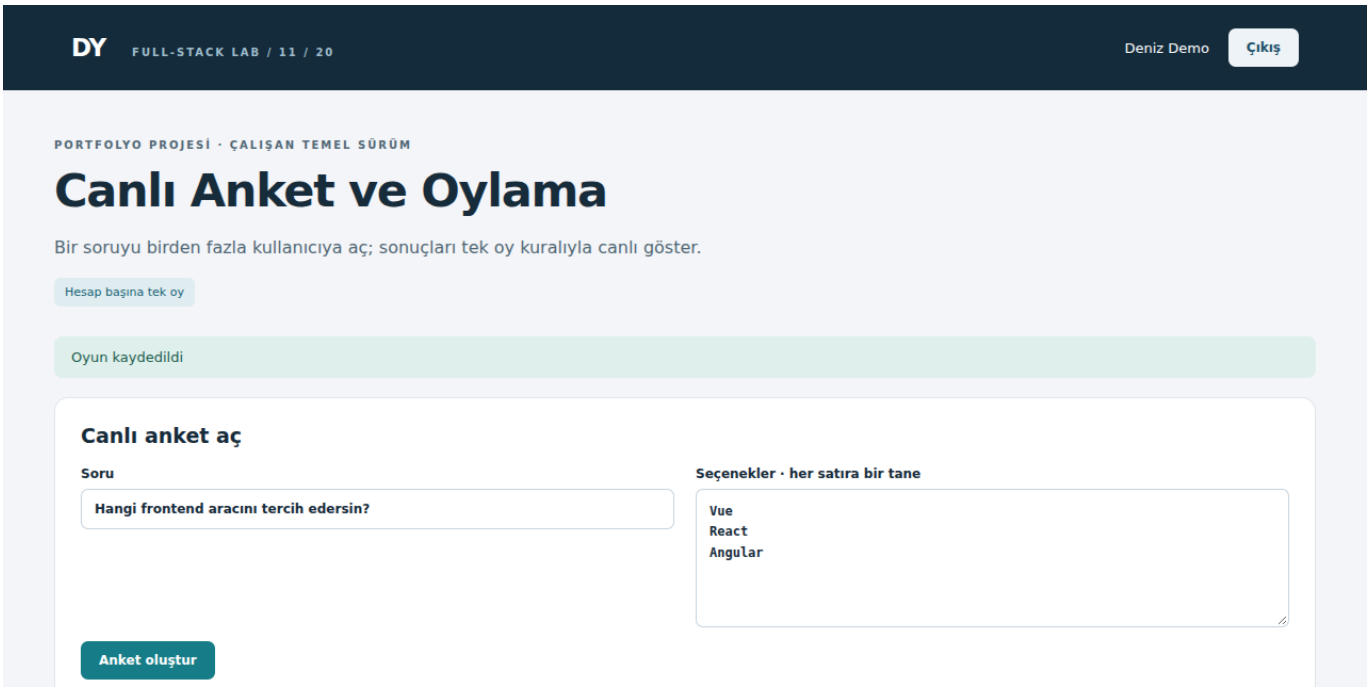
Canlı Anket ve Oylama

Bir soruyu birden fazla kullanıcıya aç; sonuçları tek oy kuralıyla canlı göster.

Hesap başına tek oy

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.



Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

2-6 seçenekli anket · Oy verme · UNIQUE ile tekrar engeli · Canlı sonuç çubukları · Anketi kapatma/açma

Bu sürümde olmayanlar

Giriş yapan kullanıcılar bütün anketleri görebilir; özel oda yok. Anonim oy, sunum kodu, oy değiştirme ve kimlik doğrulanmış kişi başına tek oy garantisi yok.

02 / Backend ve veri modeli

votes tablosunun (poll,voter) anahtarı ikinci oyu reddeder. Seçeneğin anketle ilişkisi aynı transaction içinde kontrol edilir. Kapalı ankete oy verilmez. Sonuç options LEFT JOIN votes ile sıfır oylu seçenekler dahil sayılır. SSE poll olayında istemci yeni toplamları alır.

Gerçek SQL şeması

```
CREATE TABLE polls(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),question TEXT,open
 INTEGER DEFAULT 1);


 CREATE TABLE options(id TEXT PRIMARY KEY,poll TEXT REFERENCES polls(id),label TEXT);


 CREATE TABLE votes(poll TEXT REFERENCES polls(id),voter TEXT REFERENCES users(id),option
 TEXT REFERENCES options(id),PRIMARY KEY(poll,voter));
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametreli SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/polls	Gövde yok	200
POST /api/polls	JSON: question, options	201
POST /api/polls/:id/vote	JSON: option	201
PATCH /api/polls/:id	JSON: open	200

Somut istek örneği

```
POST /api/polls
{"question":"Tercihin?","options":["Vue","Angular"]}
201 {"id":"<poll-id>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- İkinci oy 409 dönmeli ve sayımı artırmamalı.
- İki farklı hesap iki oy oluşturabilmeli.
- Kapalı anket oy kabul etmemeli.
- İlişkisiz seçenek kimliğiyle oy verilememeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const p=await post('/polls',{question:'Best?',options:['A','B']},201);const opts=(await
get('/polls'))[0].results;
await post('/polls/'+p.id+'/vote',{option:opts[0].id},201);await
post('/polls/'+p.id+'/vote',{option:opts[1].id},409);
await call('/api/polls/'+p.id+'/vote','POST',{option:opts[1].id},201,other);
assert.equal((await get('/polls'))[0].results.reduce((n,r)=>n+r.count,0),2);
await call('/api/polls/'+p.id,'PATCH',{open:false},200);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

İki kullanıcı ile farklı seçeneklere oy ver. Aynı hesapla ikinci oy isteğini API’den deneyip 409 yanıtını göster.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Özel oda ve katılım kodu ekle
- 2. Anket açılış/kapanış zamanı ekle
- 3. Sunum ekranı oluştur
- 4. Yoğun trafik için sayaç/aggregation stratejisini ölç

Anlamlı commit sırası

```
git add .
git commit -m "chore: add live-polls baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Bir kullanıcı başına tek oy ile bir insan başına tek oy aynı mıdır? Yoğun trafikte COUNT sorgusunu nasıl azaltırsın?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.